

VULNERABILITY ASSESSMENT REPORT

WEB APPLICATION SECURITY ASSESSMENT

Target: testphp.vulnweb.com
Prepared by: Laiken Naidoo

EXECUTIVE SUMMARY

A security assessment was conducted on the publicly accessible web application hosted at testphp.vulnweb.com. The objective of this assessment was to identify potential security vulnerabilities using passive and non-intrusive testing methodologies.

The assessment focused on identifying misconfigurations, information disclosure risks, and weaknesses that could potentially be leveraged in web-based attacks. Industry-standard security tools were used to analyse both network exposure and application-layer behaviour. The testing identified configuration-related vulnerabilities, including missing anti-CSRF protections and information disclosure through HTTP headers. While no critical vulnerabilities were identified within the defined scope, remediation is recommended to strengthen the application's overall security posture and reduce potential attack vectors. Overall, the risk exposure is considered moderate and primarily associated with security hardening and configuration improvements.

SCOPE & METHODOLOGY

SCOPE OF ASSESSMENT

- Passive security testing only
- No active exploitation performed
- No denial-of-service testing
- No brute-force or credential attacks
- Assessment limited to publicly accessible web components

METHODOLOGY & TOOLS USED

The assessment was conducted using industry-standard security tools to analyse both network exposure and application-layer behaviour.

Tools Used:

- Nmap – Network port scanning and service enumeration
- OWASP ZAP – Web application vulnerability identification
- Mozilla Firefox – Proxy-based traffic analysis

Testing focused on identifying configuration weaknesses, information disclosure risks, and missing security controls.

SECURITY FINDINGS

Finding 1: Absence of Anti-CSRF Protection Risk Classification: **Medium**

What We Found

The application does not implement Anti-Cross-Site Request Forgery (CSRF) tokens within its web forms. CSRF tokens are security mechanisms designed to ensure that requests submitted to the server originate from legitimate users and not from malicious third-party sources.

Why This Matters

Without CSRF protection, authenticated users could be tricked into performing unintended actions through malicious websites. In a real-world production environment, this could lead to unauthorized account changes, data manipulation, or fraudulent transactions.

This increases the application's exposure to unauthorized actions and potential abuse.

How to Fix It

- Implement unique CSRF tokens for all state-changing requests
- Validate tokens server-side before processing form submissions
- Use secure frameworks that provide built-in CSRF protection mechanisms

Absence of Anti-CSRF Tokens
URL: http://testphp.vulnweb.com/
Risk:  Medium
Confidence: Low
Parameter:
Attack:
Evidence: <form action="search.php?test=query" method="post">
CWE ID: 352
WASC ID: 9
Source: Passive (10202 - Absence of Anti-CSRF Tokens)
Input Vector:
Description:
No Anti-CSRF tokens were found in a HTML submission form.
A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL/form actions in a repeatable way. The
Other Info:
No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "goButton" "searchFor"].
Solution:
Phase: Architecture and Design
Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.
For example, use anti-CSRF packages such as the OWASP CSRFGuard.

Figure 1: OWASP ZAP identifying absence of Anti-CSRF tokens.

Finding 2: Information Disclosure via HTTP Headers

Risk Classification: **Low**

What We Found

The server discloses backend technology information through HTTP response headers, specifically the “X-Powered-By” header. This header reveals details about the underlying server technology used to run the application.

Why This Matters

While this does not directly compromise the application, exposing backend technology details can assist attackers in identifying known vulnerabilities associated with that specific software stack. Reducing publicly exposed system information is considered a security best practice and helps minimize targeted attack opportunities.

How to Fix It

- Remove or suppress the “X-Powered-By” header
- Disable unnecessary HTTP response headers in production environments
- Apply secure server configuration standards

Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)

URL: http://testphp.vulnweb.com/

Risk:  Low

Confidence: Medium

Parameter:

Attack:

Evidence: X-Powered-By: PHP/5.6.40-38+ubuntu20.04.1+deb.sury.org+1

CWE ID: 497

WASC ID: 13

Source: Passive (10037 – Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s))

Input Vector:

Description:

The web/application server is leaking information via one or more "X-Powered-By" HTTP response headers. Access to such information may facilitate attackers identifying other frameworks/components your web application is reliant upon and the vulnerabilities such components may be subject to.

Other Info:

Solution:

Ensure that your web server, application server, load balancer, etc. is configured to suppress "X-Powered-By" headers.

Figure 2: OWASP ZAP identifying server technology disclosure via HTTP headers.

RISK OVERVIEW

RISK LEVEL	NUMBER OF FINDINGS
High	0
Medium	3
Low	3
Informational	3

The security assessment identified three medium-risk and three low-risk vulnerabilities. No high-risk issues were detected within the defined scope of testing. Additionally, several informational findings were observed relating to application configuration and structure.

Overall, the application demonstrates no critical security flaws; however, improvements in security headers and input protection mechanisms are recommended to reduce exposure to common web-based attacks.

RECOMMENDATIONS AND NEXT STEPS

The following recommendations are provided to address the identified vulnerabilities and strengthen the overall security posture of the application.

1. Implement Anti-CSRF Protection

Cross-Site Request Forgery protection should be implemented by including unique, unpredictable tokens within all state-changing requests. This will prevent unauthorised commands from being executed on behalf of authenticated users.

2. Configure Security Headers

The following HTTP security headers should be properly configured:

- Content-Security-Policy (CSP)
- X-Frame-Options (Anti-clickjacking)
- X-Content-Type-Options
- Removal of unnecessary "X-Powered-By" and "Server" headers

Proper header configuration will mitigate risks such as clickjacking, MIME-type sniffing, and information disclosure.

3. Address Potential XSS Risks

All user-controllable inputs should be validated and sanitised. Output encoding should be applied where necessary to prevent injection of malicious scripts into the application.

4. Conduct Regular Security Testing

It is recommended that regular vulnerability assessments and penetration testing be conducted to ensure ongoing security compliance and early detection of emerging threats.

By implementing the above recommendations, the organisation can significantly reduce its exposure to common web-based attacks and enhance overall application security. Continuous monitoring and proactive security practices are essential in maintaining a secure web environment.